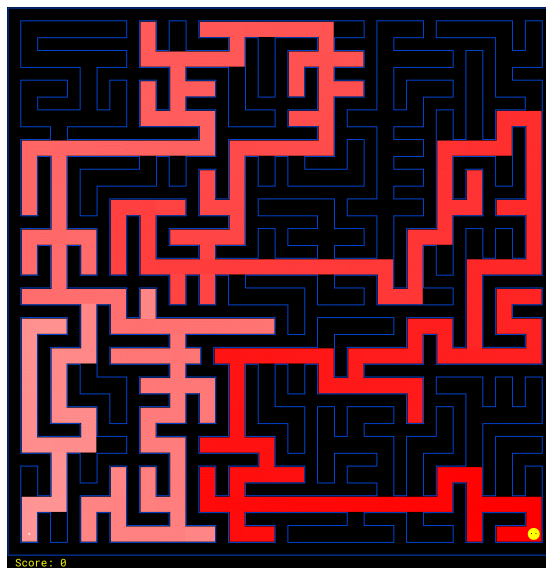


## Project 1: Search in Pac-Man



All those colored walls,  
Mazes give Pac-Man the blues,  
So teach them to search.

### Introduction

In this project, your Pac-Man agent will find paths through their maze world, both to reach a particular location and to collect food efficiently. You will build general search algorithms and apply them to different Pac-Man scenarios.

The code for this project consists of several Python files, some of which you will need to read and understand in order to complete the assignment, and some you can glance over.

### Submission

You will fill in portions of `pacai/student/search.py` and `pacai/student/searchAgents.py` during this assignment. You should **only** submit these two files.

This assignment should be submitted with the filename `solution.zip` [HERE](#). Please use the command line `zip` tool to make sure your submission gets zipped correctly:

```
zip -j solution.zip pacai/student/search.py pacai/student/searchAgents.py
```

For these submissions, unzip should directly yield the source files and **not** a directory named "solution", "pN", or "pacai". Note that this is counter to standard conventions when sending a zip file to a human, but easier for the autograder.

After submitting, the autograder will grade your submission and report the result back to you. Do not close the tab or you will not be able to see your score. You can submit as many times as you want. However, if we find you continually making tiny changes instead of testing locally, then we will deduct points. Any attempt to trick the autograder is considered cheating.

### Evaluation

Your code will be autograded for technical correctness. Please *do not* change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder (and points will be deducted!). However, you are allowed to add any new classes or function that you need. The correctness of your implementation -- not the autograder's output -- will be the final judge of your score. If necessary, we will review and grade assignments individually to ensure that you receive due credit for your work. This assignment is graded out of 25 points. 20 points will be for correctness as determined by the autograder and the point system given below for each problem. 5 points will be for style, which the autograder will also check. You can run the style checker using the `run_style.sh` script in the project root.

### Academic Dishonesty

We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else's code and submit it with minor changes, we will know. These cheat detectors are quite hard to fool, so please don't try. We trust you all to submit your own work only; *please* don't let us down. If you do, we will pursue the strongest consequences available to us.

### Getting Help

You are not alone! If you find yourself stuck on something, contact the course staff for help. Office hours, section, and Piazza are there for your support; please use them. If you can't make our office hours, let us know and we will schedule more. We want these projects to be rewarding and instructional, not frustrating and demoralizing. But, we don't know when or how to help unless you ask. One more piece of advice: if you don't know what a variable does or what kind of values it takes, print it out.

### Running Code on Your Local Machine

In order to run the Pac-Man code on your local machine, [you must have Tk](#), python  $\geq 3.5$ , and pip installed. Finally you want to install Pacai's package requirements listed in the `requirements.txt` file in the project's root directory:

```
pip3 install --user -r requirements.txt
```

For the next set of instructions, simply follow the steps listed below depending on your OS.

#### Linux

Install the Python binding for the Tk package, usually called something like `python3-tk`. On Ubuntu you can use the following command:

```
sudo apt-get install python3-tk
```

**Mac OS X**

The required additional components (Tk and Python3 Tk bindings) are typically bundled with the Python3 package.

**Windows**

There are two separate methods for getting Pacai to work on Windows. One is using the [Windows Subsystem for Linux](#) (WSL) and the other is using [Git Bash](#). We will first talk about the WSL.

**Windows Subsystem for Linux (WSL)**

1. First, make sure you have the WSL installed on your local machine. You can follow [this installation guide](#). You are allowed to pick any distribution, however we will be providing specific instructions for Ubuntu. Other distributions will have similar steps.

2. Next, you want to make sure you have an X server running. This allows the WSL to launch graphical applications. We recommend using VcXsrv and following [this installation guide](#). Make sure your X server is running whenever you want to run Pacai with graphics.

3. Now launch your WSL. You can do this by typing "Windows Subsystem for Linux" in your start menu and clicking on the icon.

4. Next, you must configure your bash inside the WSL to use the X server from step #2. To do this, use the following commands:

```
echo "export DISPLAY=localhost:0.0" >> ~/.bashrc
source ~/.bashrc
```

This sets your `DISPLAY` environmental variable in your `bashrc` and then reloads your `bashrc`.

5. Let's make sure you have all the required packages:

```
sudo apt-get update
sudo apt-get install python3 python3-pip python3-tk x11-apps
```

7. In order to test that you have the graphics set up correctly, you can use the `xeyes` command:

```
xeyes
```

8. Finally, clone the Pacai repository and install Pacai's package requirements:

```
pip3 install --user -r requirements.txt
```

**Git Bash**

1. Install [Git Bash](#), you can follow [this installation guide](#).

2. Now launch your Git Bash. You can do this by typing "Git Bash" in your start menu and clicking on the icon.

3. If you are using a newer version of Windows, there may be a conflict with the version of Python 3 installed from the Windows Store. This conflict will cause a permission denied error when running Python 3 in Git Bash. To avoid this error, you will want to disable the Windows Store version of Python. To do this, type "manage app execution aliases" into your start menu and click on the icon. Within the app execution aliases, disable both `python.exe` and `python3.exe`.

4. You may also want to create an alias for `python3`. All the commands in the instructions use `python3`, but Git Bash just refers to the executable as `python`. To create the alias, you can use the following commands:

```
echo "alias python3=python" >> ~/.bash_profile
source ~/.bash_profile
```

5. Finally, clone the Pacai repository and install Pacai's package requirements:

```
pip3 install --user -r requirements.txt
```

**Code**

All the code for this (and later projects) is available in this repository: <https://github.com/lings/pacman>. The only files you should edit are located in the [pacai.student](#) package. You should **not** use any third-party libraries, but the [Python Standard Library](#) is fair-game. If a bug is found in the code (non-student code, then the class will be alerted and you will have to pull the changes from this repository.

Any commands provided throughout these instructions are to be executed from the project root directory (the one with the `README.md` and `LICENSE.md` files).

There are many files that will be used throughout this quarter-long project. Below are a few that you should focus on for this assignment.

- Agents
  - [pacai.agents.base.BaseAgent](#)
  - [pacai.agents.search.base.SearchAgent](#)
- Actions / Directions
  - [pacai.core.directions.Directions](#)
  - [pacai.core.actions.Actions](#)
- States
  - [pacai.core.gamestate.AbstractGameState](#)
  - [pacai.bin.pacman.PacmanGameState](#)
  - [pacai.core.agentstate.AgentState](#)
- Search
  - [pacai.core.search.problem.SearchProblem](#)
  - [pacai.core.search.position.PositionSearchProblem](#)
  - [pacai.core.search.food.FoodSearchProblem](#)
- Data Structures
  - [pacai.util.stack](#)
  - [pacai.util.queue](#)
  - [pacai.util.priorityQueue](#)

**Welcome to Pac-Man**

Welcome to Pac-Man's world, wakka wakka!

Let's jump right into the assignment and play a game with Pac-man! To begin a game of Pac-Man, use the following command:

```
python3 -m pacai.bin.pacman
```

Pac-Man lives in a shiny blue world of twisting corridors and tasty round treats. Navigating this world efficiently will be Pac-Man's first step in mastering their domain. Play a quick game and hopefully... Pac-Man emerges victorious! Lets try and continue this trend of victories throughout the rest of the quarter.

**Note:** If you are having errors regarding python-tk, use your system's package manager to install *python-tk*. [Here](#) is a link to more detailed instructions, if needed.

### First Agent

Since we won't always have direct control over Pac-Man, lets start working with agents that will control them for us. The simplest agent, [GoWestAgent](#), which always goes West (a trivial reflex agent). This agent can occasionally win:

```
python3 -m pacai.bin.pacman --layout testMaze --pacman GoWestAgent
```

But, things get ugly for this agent when turning is required:

```
python3 -m pacai.bin.pacman --layout tinyMaze --pacman GoWestAgent
```

If (when) Pac-Man gets stuck, you can exit the game by typing CTRL-c into your terminal. Soon, your agent will solve not only `tinyMaze`, but any maze you want. Note that `pacai.bin.pacman` supports a number of options that can each be expressed in a long way (e.g., `--layout`) or a short way (e.g., `-l`). You can see the list of all options and their default values via:

```
python3 -m pacai.bin.pacman --help
```

## Finding a Fixed Food Dot using Search Algorithms

The [SearchAgent](#) class plans out a path through Pac-Man's world and then executes that path step-by-step. The search algorithms for formulating a plan are not implemented -- this is your job. Before you begin your implementation test that the [SearchAgent](#) is working correctly by running:

```
python3 -m pacai.bin.pacman --layout tinyMaze --pacman SearchAgent --agent-args fn=pacai.core.search.search.tinyMazeSearch
```

The command above tells the [SearchAgent](#) to use [tinyMazeSearch](#) as its search algorithm. Pac-Man should navigate this maze successfully, however, will fail and even perform illegal actions on other layouts:

```
python3 -m pacai.bin.pacman --layout mediumMaze --pacman SearchAgent --agent-args fn=pacai.core.search.search.tinyMazeSearch
```

This is because [tinyMazeSearch](#) is hand made, catered to tiny maze. Therefore, [tinyMazeSearch](#) is not a generic search function and will not work on every maze.

With that in mind, it's time to write full-fledged generic search functions to help Pac-Man plan routes! Pseudocode for the search algorithms you'll write can be found in the lecture slides and textbook. Remember that a search node must contain not only a state but also the information necessary to reconstruct the path (plan) which gets to that state.

**Important note:** All of your search functions need to return a list of *actions* that will lead the agent from the start to the goal. These actions all have to be legal moves (valid directions, no moving through walls).

**Hint:** Each algorithm is very similar. Algorithms for DFS, BFS, UCS, and A\* differ only in the details of how the fringe is managed. So, concentrate on getting DFS right and the rest should be relatively straightforward. Indeed, one possible implementation requires only a single generic search method which is configured with an algorithm-specific queuing strategy. (Your implementation need *not* be of this form to receive full credit).

**Hint:** Make sure to check out the classes in [pacai.util.stack](#), [pacai.util.queue](#), and [pacai.util.priorityQueue](#).

### Question 1 (2 points)

Implement the depth-first search (DFS) algorithm in [pacai.student.search.depthFirstSearch](#). To make your algorithm *complete*, write the graph search version of DFS, which avoids expanding any already visited states (textbook section 3.5).

Your code should quickly find a solution for:

```
python3 -m pacai.bin.pacman --layout tinyMaze --pacman SearchAgent --agent-args fn=pacai.student.search.depthFirstSearch
```

```
python3 -m pacai.bin.pacman --layout mediumMaze --pacman SearchAgent --agent-args fn=pacai.student.search.depthFirstSearch
```

```
python3 -m pacai.bin.pacman --layout bigMaze --pacman SearchAgent --agent-args fn=pacai.student.search.depthFirstSearch
```

The Pac-Man board will show an overlay of the states explored, and the order in which they were explored (deeper red means earlier exploration). Is the exploration order what you would have expected? Does Pac-Man actually go to all the explored squares on their way to the goal?

**Hint:** If you use a [Stack](#) as your data structure, the solution found by your DFS algorithm for `mediumMaze` should have a length of 130. If you are getting 244, you pushed the successors onto the fringe in the reverse order provided by [Actions.getSuccessor\(\)](#). Is this a least cost solution? If not, think about what depth-first search is doing wrong.

### Question 2 (1 point)

Implement the breadth-first search (BFS) algorithm in [pacai.student.search.breadthFirstSearch](#). Again, write a graph search algorithm that avoids expanding any already visited states. Test your code the same way you did for depth-first search.

```
python3 -m pacai.bin.pacman --layout mediumMaze --pacman SearchAgent --agent-args fn=pacai.student.search.breadthFirstSearch
```

```
python3 -m pacai.bin.pacman --layout bigMaze --pacman SearchAgent --agent-args fn=pacai.student.search.breadthFirstSearch
```

Does BFS find a least cost solution? If not, check your implementation.

**Hint:** If Pac-Man moves too slowly for you, increase the fps with `--fps 50`.

**Note:** If you've written your search code generically, your code should work equally well for the eight-puzzle search problem (textbook section 3.2) without any changes.

```
python3 -m pacai.bin.eightpuzzle
```

## Varying the Cost Function

While BFS will find a fewest-actions path to the goal, we might want to find paths that are "best" in other senses. Consider `mediumDottedMaze` and `mediumScaryMaze`. By changing the cost function, we can encourage Pac-Man to find different paths. For example, we can charge more for dangerous steps in ghost-ridden areas or less for steps in food-rich areas, and a rational Pac-Man agent should adjust its behavior in response.

### Question 3 (2 points)

Implement the uniform-cost graph search algorithm in [pacai.student.search.uniformCostSearch](#). You should now observe successful behavior in all three of the following layouts. Here the agents are all UCS agents that differ only in the cost function they use (the agents and cost functions are written for you):

```
python3 -m pacai.bin.pacman --layout mediumMaze --pacman SearchAgent --agent-args fn=pacai.student.search.uniformCostSearch
```

```
python3 -m pacai.bin.pacman --layout mediumDottedMaze --pacman StayEastSearchAgent
```

```
python3 -m pacai.bin.pacman --layout mediumScaryMaze --pacman StayWestSearchAgent
```

Note: Both [StayEastSearchAgent](#) and [StayWestSearchAgent](#) use [uniformCostSearch](#).

Note: You should get low and high path costs for [StayEastSearchAgent](#) and [StayWestSearchAgent](#) respectively. This is due to their exponential cost functions.

## A\* search

### Question 4 (3 points)

Implement A\* graph search algorithm in [pacai.student.search.aStarSearch](#). A\* takes a heuristic function as an argument. The heuristics function take two arguments: a state in the search problem (the main argument), and the problem itself (for reference information). The [null heuristic](#) is a trivial example.

You can test your A\* implementation on the original problem of finding a path through a maze to a fixed position using the Manhattan distance heuristic. This heuristic is implemented for you already as [pacai.core.search.heuristic.manhattan](#).

```
python3 -m pacai.bin.pacman --layout bigMaze --pacman SearchAgent --agent-args
fn=pacai.student.search.aStarSearch,heuristic=pacai.core.search.heuristic.manhattan
```

You should see that A\* finds the optimal solution slightly faster than uniform cost search. From around 619 nodes expanded to 538 nodes (ties in priority may vary output). What happens on `openMaze` for the various search strategies?

## Finding All the Corners

The real power of A\* will only be apparent with a more challenging search problem. Now, it's time to formulate a new problem and design a heuristic for it.

In *corner mazes*, there are four dots, one in each corner. Our new search problem is to find the shortest path through the maze that touches all four corners (whether the maze actually has food there or not). Note that for some mazes like `tinyCorners`, the shortest path does not always go to the closest food first! *Hint*: the shortest path through `tinyCorners` takes 28 steps.

### Question 5 (2 points)

Implement the search problem [pacai.student.searchAgents.CornersProblem](#). You will need to choose a **state representation** that encodes all the information necessary to detect whether all four corners have been reached. Now, your search agent should solve:

```
python3 -m pacai.bin.pacman --layout tinyCorners --pacman SearchAgent --agent-args
fn=pacai.student.search.breadthFirstSearch,prob=pacai.student.searchAgents.CornersProblem
```

```
python3 -m pacai.bin.pacman --layout mediumCorners --pacman SearchAgent --agent-args
fn=pacai.student.search.breadthFirstSearch,prob=pacai.student.searchAgents.CornersProblem
```

To receive full credit, you need to define an abstract state representation that *does not* encode irrelevant information (like the position of ghosts, where extra food is, etc.). In particular, do not use a Pac-Man [PacmanGameState](#) as a search state. Your code will be very, very slow if you do (and also wrong).

*Hint*: The only parts of the game state you need to reference in your implementation are the starting Pac-Man position and the location of the four corners.

Our implementation of [breadthFirstSearch](#) expands just under 2000 search nodes on `mediumCorners`. However, heuristics (used with A\* search) can reduce the amount of searching required.

### Question 6 (3 points)

Implement a heuristic for [CornersProblem](#) in [pacai.student.searchAgents.cornersHeuristic](#). To test your code run:

```
python3 -m pacai.bin.pacman --layout mediumCorners --pacman AStarCornersAgent
```

Note: [pacai.core.search.corners.AStarCornersAgent](#) is an agent with these command line arguments:

```
--pacman SearchAgent --agent-args
fn=pacai.student.search.aStarSearch,prob=pacai.student.searchAgents.CornersProblem,heuristic=pacai.student.searchAgents.cornersHeuristic
```

Grading: inadmissible heuristics will get *no* credit.

- 1 point for **any** admissible heuristic
- 2 point for expanding fewer than **1600** nodes
- 3 point for expanding fewer than **1200** nodes
- Expand fewer than **800**, and you're doing great!

*Hint*: Remember, heuristic functions just return numbers, which, to be admissible, must be lower bounds on the actual shortest path cost to the nearest goal.

## Eating All The Dots

Now we'll solve a hard search problem: eating all the Pac-Man food in as few steps as possible. For this, a new search problem definition which formalizes the food-clearing problem can be found in [pacai.core.search.food.FoodSearchProblem](#). A solution is defined to be a path that collects all of the food in the Pac-Man world.

For the present project, solutions do not take into account any ghosts or power pellets. They only depend on the placement of walls, regular food, and Pac-Man. Of course ghosts can ruin the execution of a solution! We'll get to that in the next project :) If you have written your general search methods correctly, [aStarSearch](#) with a null heuristic (equivalent to uniform-cost search) should quickly find an optimal solution to `testSearch` with no code change on your part (total cost of 7).

```
python3 -m pacai.bin.pacman --layout testSearch --pacman AStarFoodSearchAgent
```

Note: [AStarFoodSearchAgent](#) is an agent with these command line arguments:

```
--pacman SearchAgent --agent-args
fn=pacai.student.search.aStarSearch,prob=pacai.core.search.food.FoodSearchProblem,heuristic=pacai.student.searchAgents.foodHeuristic
```

You should find that [uniformCostSearch](#) starts to slow down even for the seemingly simple `tinySearch`. As a reference, our implementation takes 1.2 seconds to find a path of length 27 after expanding 4847 search nodes.

### Question 7 (5 points)

Implement a heuristic for [FoodSearchProblem](#) in [pacai.student.searchAgents.foodHeuristic](#). Try your agent on `trickySearch`:

```
python3 -m pacai.bin.pacman --layout trickySearch --pacman AStarFoodSearchAgent
```

Our [uniformCostSearch](#) agent finds the optimal solution in about 8.5 seconds, exploring over 16,000 nodes. If your heuristic is admissible, you will receive the following score, depending on how many nodes your heuristic expands.

Grading: inadmissible heuristics will get *no* credit.

- 1 point for expanding fewer than **15000** nodes (very easy)
- 2 point for expanding fewer than **12000** nodes (easy)
- 3 point for expanding fewer than **9000** nodes (medium)
- 4 point for expanding fewer than **7000** nodes (hard)

Think through admissibility carefully, as inadmissible heuristics may manage to produce fast searches and even optimal paths. Can you solve `mediumSearch` in a short time? If so, we're either very, very impressed, or your heuristic is inadmissible.

### Admissibility vs. Consistency?

Technically, admissibility isn't enough to guarantee correctness in graph search -- you need the stronger condition of consistency. For a heuristic to be consistent, it must hold that if an action has cost  $c$ , then taking that action can only cause a drop in heuristic of at most  $c$ . If your heuristic is not only admissible, but also consistent, you will receive **1 additional point** for this question.

Almost always, admissible heuristics are also consistent, especially if they are derived from problem relaxations. Therefore, it is probably easiest to start out by brainstorming admissible heuristics. Once you have an admissible heuristic that works well, you can check whether it is indeed consistent, too. Inconsistency can sometimes be detected by verifying that your returned solutions are non-decreasing in  $f$ -value. Moreover, if UCS and A\* ever return paths of different lengths, your heuristic is inconsistent. This stuff is tricky. If you need help, don't hesitate to ask the TAs!

## Suboptimal Search

Sometimes, even with A\* and a good heuristic, finding the optimal path through all the dots is hard. In these cases, we'd still like to find a reasonably good path, quickly. In this section, you'll write an agent that always eats the closest dot.

### Question 8 (2 points)

Implement the function [pacai.student.searchAgents.ClosestDotSearchAgent.findPathToClosestDot](#). Our agent solves this maze (suboptimally!) in under a second with a path cost of 350:

```
python3 -m pacai.bin.pacman --layout bigSearch --pacman ClosestDotSearchAgent
```

*Hint:* The quickest way to complete [findPathToClosestDot](#) is to complete [pacai.student.searchAgents.AnyFoodSearchProblem](#). Then, solve that problem with an appropriate search function. The solution should be very short!

Your [ClosestDotSearchAgent](#) won't always find the shortest possible path through the maze. If you don't understand why, ask the TAs! In fact, you can do better if you try.

### Mini Contest (2 points extra credit)

Implement an [pacai.student.searchAgents.ApproximateSearchAgent](#) that finds a short path through `bigSearch`. The three teams that find the shortest path using no more than 30 seconds of computation will receive extra credit points.

```
python3 -m pacai.bin.pacman --layout bigSearch --pacman ApproximateSearchAgent --null-graphics
```

We will time your agent using the no graphics option `--null-graphics`, and it must complete in under 30 seconds on our grading machines. Please describe what your agent is doing in a comment! We reserve the right to give additional extra credit to creative solutions, even if they don't work that well. Don't hard-code the path, of course.